Materials:

1. RPi Pico
2. Jumper Wires (x3)
3. Servo Motor (9G)

For Servo Motor

Red = 5Volts

Orange = Data

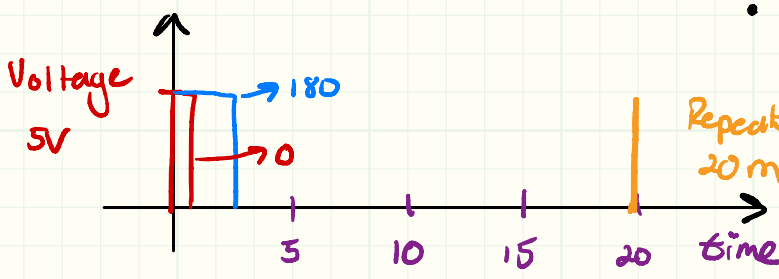
Brown = GND

Important Note:

- This is a very small servo motor
  - ↳ draws very small amount of current from the board
  - ↳ if this was longer current → would need to use an external battery
  - ↳ normally bad practice to use PiPico as Power Source
    - ↳ still need common ground

Inner Workings

- Motor goes from 0 → 180°



- The desired position depends on the width of a pulse sent onto orange pin!
  - width → time

Red = 0.5 ms → 0 degrees

Blue = 2.5 ms → 180 degrees

- Have to repeat pulse to hold in exact position

- Linear scale from 0→180° ; 0.5ms → 2.5 ms

Using Pulse Width Modulation (PWM) → Can Send Pulses

- But how does the Rpi Pico actually send Pulse?
- We want our PWM period to be 20ms (repeats)
- For PWM, we don't set a time period
  - ↳ we set a frequency! samples/sec
- If period = 20ms → what is frequency (F)?

$$f = \frac{1}{\text{period}} = \frac{1}{20 \times 10^{-3}} = 50 \text{ Hz}$$

Magic Servo PWM Freq!

How do we actually program this?

To find actual value, what percentage (%) of the period is the signal "up" or "on"?

write_value = (%) * 65,535 → Total # of numbers for 16-bit number

Where does 65,535 and 16-bit come from?

A bit let's us represent something in binary form

- 1 bit → either on or off
  - 1 or 0
- I can't represent a lot with that so I increase the bit resolution
  - 2 bit → 00 or 01 or 10 or 11
    - 4 states!

$2^{\#bits}$ → gives me my total resolution

$2^{16} = 65,536$! But actul # range 0 → 65,535!

- By saying % * 65,535 → I can find out how "long" the pulse is and control the motor!
- The (%) = time on (ms) / period (ms)

- For example:

For zero degrees → 0.5 ms / 20 ms = 0.025 = 2.5%

Now, find the actual value:

write_0 = 0.5 ms / 20 ms * 65,535 = 1,638 round down

For 180 degrees → 2.5 ms / 20 ms * 65,535 = 8191 round down

Putting it together

- If I make a PWM signal @ → frequency = 50 Hz, @ → Duty value = 1,638] 0°
  @ → frequency = 50 Hz, @ → Duty = 8,191] 180°

Linear Eq!

Example ①:

```
import machine
import time

servo = machine.PWM(machine.Pin(15)) # First, we must set the RPi Pico Pin we want to use for PWM signals

servo.freq(50) # Remember, the period is 20 ms ... freq = 1/T = 1/0.020 = 50 Hz!

while True: # Let's have the servo do some action over and over again for all time

    servo.duty_u16(1638) # We found that 1,638 corresponds to 0 degrees!

    time.sleep(1) # This is a mechanical load ... motors can't move @ MHz speeds! Have to have a delay!

    servo.duty_u16(8191) # We found that 8,191 corresponds to 180 degrees!

    time.sleep(1) # Again, if we don't delay here, the motor will never move, too fast!
```

- Now, I want to make this easy. "go to n degrees!"

In the real world, we like things EASY!

↳ "Go to N degrees!" and servo moves

↳ How can we do this?

. If I want 0 degrees (output) → I input 1638

. If I want 180 degrees (output) → I input 8191

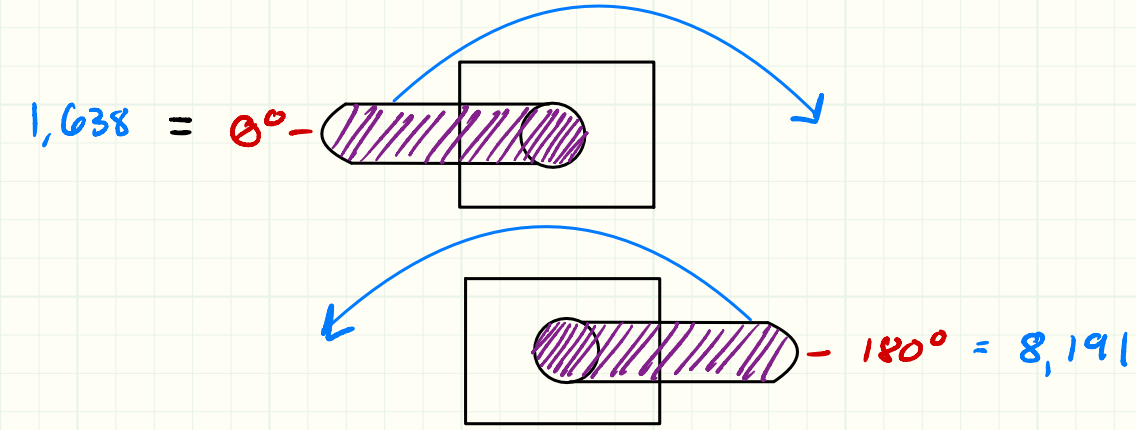
X_n = Input

Y_n = output

. I can treat this as a linear equation!

↳ If I vary by some # → I can slowly change Angles!

$X_1 = 0°$; $y_1 = 1,638$; $X_2 = 180°$; $y_2 = 8,191$



Remember from Algebra 2 → Equation of a Line

Point - Slope Formula - Represent specific point on Line

$$y - y_1 = m(x - x_1)$$

$$\text{Slope } (m) = \frac{y_2 - y_1}{x_2 - x_1} = \frac{8191 - 1638}{180 - 0} = \left[\frac{6,553}{180}\right]$$

$$y - 1,638 = \left(\frac{6,553}{180}\right)(x - 0) \rightarrow y = \frac{6,553}{180}x + 1,638$$

Magic formula → Inserting desired angle as (X) gives us the duty number to input! $y = \left(\frac{6,553}{180}\right)X + 1,638$ (1,638: y-intercept)

↳ Therefore: $write\_value = \left(\frac{6553}{180}\right)(\theta) + 1{,}638$

θ (theta) = degrees (angle)

Example (2):

```
1  import machine
2  import time
3
4  servo = machine.PWM(machine.Pin(15)) # First, we must set the RPi Pico Pin we want to use for PWM signals
5
6  servo.freq(50) # Remember, the period is 20 ms ... freq = 1/T = 1/0.020 = 50 Hz!
7
8  while True: # Let's have the servo do some action over and over again for all time
9
10     # Let's grab an input from the user for the angle of the motor
11     angle = int(input('Please Enter A Desired Angle'))
12
13     # Now, we input our Linear Equation to determine the appropriate duty value!
14     duty_value = (6553/180)*angle + 1638
15
16     # Now, we have our duty value, but needs to be an integer before going to motor
17     servo.duty_u16(int(duty_value))
18
19     # We need to always add some kind of delay since this is a mechanical load!
20     time.sleep(0.5)
```

: "spin" makes clear

Note: Do NOT Fidget with (Spin) Servo Horn/Gears

↳ Will Strip gears!